Cubist Data Project: Raphaël Bahisson

Question 1: Thinknum

Thinknum is an alternative dataset provider for US equities. Its datasets are gathered from publicly available online sources such as company websites, product princing pages or job listings. Thinknum presents multiple characteristics that make it perfectly fit to be used for the sector of systematic trading.

First of all, Thinknum standardizes and tracks any kind of changes in the data over time, creating high-frequency, continuously updated datasets. This allows traders to capture trends earlier rather than if they were only basing themselves on lagged financial statements. Moreover, the data is structured, timestamped and accurate at any point in time. Traders can then use it to backtest strategies without having to fear for look-ahead bias.

Additionally, Thinknum tracks traditional data as well as completely unique signals. For example, it also looks at a company's hiring activity, inventory changes or proxies for consumer demand. These are all indicators closely correlated to the overall performance of a company before the release of its financial statements. This data gives yet another edge to systematic traders when they have to define their future strategy.

Finally, the data is directly accessible via API, which makes its implementation within models fast and efficient.

Question 2: Webscrape

Save the Data into a Pandas DataFrame

```
import requests
import json
import time
import os
import pandas as pd
from datetime import datetime
from dateutil.relativedelta import relativedelta
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
```

```
raw_data_dir = "raw_data"
os.makedirs(raw_data_dir, exist_ok=True)

security_types = ["Bill", "Note"]

start_date = "2022-01-01"
end_date = "2022-12-31"
```

```
def fetch_auction_data(security_type, start_date, end_date):

    all_records = []
    current     = datetime.strptime(start_date, "%Y-%m-%d")
    end         = datetime.strptime(end_date,   "%Y-%m-%d")

    base_url = "https://www.treasurydirect.gov/TA_WS/securities/search"

    while current <= end:
        month_start = current.strftime("%Y-%m-%d")
        month_end   = (current + relativedelta(months=1) - relativedelta(day

        if datetime.strptime(month_end, "%Y-%m-%d") > end:
            month_end = end_date

        params = {
            "type":          security_type,
            "dateFieldName": "auctionDate",
            "startDate":     month_start,
            "endDate":       month_end,
            "format":        "json",         }

        response = requests.get(base_url, params=params, timeout=30)
        response.raise_for_status()
        records = response.json()

        all_records.extend(records)

        current += relativedelta(months=1)
        time.sleep(0.3)

    return all_records
```

```
raw_data = {}

for sec_type in security_types:
    records = fetch_auction_data(sec_type, start_date, end_date)
    raw_data[sec_type] = records

    filename = f"{raw_data_dir}/{sec_type.lower()}s_2022_raw.json"
    with open(filename, "w") as f:
        json.dump(records, f, indent=2)
```

```
all_records = []
```

```
for sec_type, records in raw_data.items():
    for record in records:
        record["_security_type"] = sec_type
        all_records.append(record)

df_raw = pd.DataFrame(all_records)

print(f"\nTotal records before filtering: {len(df_raw)}")
print(f"Columns: {df_raw.columns.tolist()}\n")
print("Unique values in 'type' column:")
print(df_raw["type"].value_counts())
```

```
Total records before filtering: 335
Columns: ['cusip', 'issueDate', 'securityType', 'securityTerm', 'maturityDate

Unique values in 'type' column:
type
Bill    232
Note     60
CMB      43
Name: count, dtype: int64
```

```
df_raw.shape
```

```
(335, 121)
```

```
# Dropping all empty columns

df_clean = df_raw.dropna(axis=1, how="all")
df_clean = df_clean.loc[:, (df_clean != "").any()]
df_clean = df_clean[df_clean["type"].isin(["Bill", "Note"])].reset_index(dro

df_clean.shape
```

```
(292, 95)
```

```
df_clean
```

	cusip	issueDate	securityType	securityTerm	maturityDate	intere
0	912796P45	2022-02-03T00:00:00	Bill	13-Week	2022-05-05T00:00:00	
1	912796S67	2022-02-03T00:00:00	Bill	26-Week	2022-08-04T00:00:00	
2	912796S26	2022-02-01T00:00:00	Bill	4-Week	2022-03-01T00:00:00	
3	912796T25	2022-02-01T00:00:00	Bill	8-Week	2022-03-29T00:00:00	
4	912796S34	2022-01-27T00:00:00	Bill	52-Week	2023-01-26T00:00:00	
...	...	...	...	...	...	
287	91282CGB1	2023-01-03T00:00:00	Note	7-Year	2029-12-31T00:00:00	3.
288	91282CGC9	2023-01-03T00:00:00	Note	5-Year	2027-12-31T00:00:00	3.
289	91282CGD7	2023-01-03T00:00:00	Note	2-Year	2024-12-31T00:00:00	4.
290	91282CGA3	2022-12-15T00:00:00	Note	3-Year	2025-12-15T00:00:00	4.
291	91282CFV8	2022-12-15T00:00:00	Note	9-Year 11-Month	2032-11-15T00:00:00	4.

292 rows × 95 columns

Data Types for each column:

```
print(df_raw.columns)
```

```
Index(['cusip', 'issueDate', 'securityType', 'securityTerm', 'maturityDate',
       'interestRate', 'refCpiOnIssueDate', 'refCpiOnDatedDate',
       'announcementDate', 'auctionDate',
       ...
       'unadjustedPrice', 'updatedTimestamp', 'xmlFilenameAnnouncement',
       'xmlFilenameCompetitiveResults', 'xmlFilenameSpecialAnnouncement',
       'tintCusip1', 'tintCusip2', 'tintCusip1DueDate', 'tintCusip2DueDate',
       '_security_type'],
      dtype='object', length=121)
```

The API gave us the data structured over 121 columns. For clarity, we class them into the following 6 groups:

- datetime $\longrightarrow$ date and timestamps
- boolean $\longrightarrow$ Bernoulli variables (Yes/No strings)

- int64 → whole numbers
- float → numbers that include decimals: rates, prices, amounts, ratios
- category → text fields
- string → identifiers and filenames

Regarding data keys, the *'cusip'* column identifies each row in the dataset thanks to a 9-character code, which is unique to each row in the dataset

```
date_cols = [
    'issueDate', 'maturityDate', 'announcementDate', 'auctionDate',
    'datedDate', 'backDatedDate', 'callDate', 'calledDate',
    'firstInterestPaymentDate', 'frnIndexDeterminationDate', 'maturingDate',
    'originalDatedDate', 'originalIssueDate', 'updatedTimestamp',
    'tintCusip1DueDate', 'tintCusip2DueDate' ]

bool_cols = [
    'backDated', 'callable', 'cashManagementBillCMB', 'fimaIncluded',
    'floatingRate', 'reopening', 'somaIncluded', 'strippable', 'tips',
    'competitiveTendersAccepted', 'noncompetitiveTendersAccepted' ]

int_cols = [
    'auctionDateYear', 'allocationPercentageDecimals', 'competitiveBidDecima
    'interestPaymentFrequency', 'minimumBidAmount', 'minimumStripAmount',
    'minimumToIssue', 'multiplesToBid', 'multiplesToIssue',
    'nlpExclusionAmount', 'nlpReportingThreshold',
    'maximumCompetitiveAward', 'maximumNoncompetitiveAward', 'maximumSingleB

float_cols = [
    'interestRate', 'refCpiOnIssueDate', 'refCpiOnDatedDate',
    'accruedInterestPer1000', 'accruedInterestPer100',
    'adjustedAccruedInterestPer1000', 'adjustedPrice', 'allocationPercentage
    'averageMedianDiscountRate', 'averageMedianInvestmentRate',
    'averageMedianPrice', 'averageMedianDiscountMargin', 'averageMedianYield
    'bidToCoverRatio', 'competitiveAccepted', 'competitiveTendered',
    'currentlyOutstanding', 'directBidderAccepted', 'directBidderTendered',
    'estimatedAmountOfPubliclyHeldMaturingSecuritiesByType',
    'fimaNoncompetitiveAccepted', 'fimaNoncompetitiveTendered',
    'frnIndexDeterminationRate', 'highDiscountRate', 'highInvestmentRate',
    'highPrice', 'highDiscountMargin', 'highYield', 'indexRatioOnIssueDate',
    'indirectBidderAccepted', 'indirectBidderTendered',
    'lowDiscountRate', 'lowInvestmentRate', 'lowPrice',
    'lowDiscountMargin', 'lowYield', 'noncompetitiveAccepted',
    'offeringAmount', 'pricePer100', 'primaryDealerAccepted',
    'primaryDealerTendered', 'somaAccepted', 'somaHoldings', 'somaTendered',
    'spread', 'standardInterestPaymentPer1000', 'tiinConversionFactorPer1000
    'totalAccepted', 'totalTendered', 'treasuryRetailAccepted',
    'treasuryRetailTendersAccepted', 'unadjustedAccruedInterestPer1000',
    'unadjustedPrice' ]

cat_cols = [
    'securityType', 'securityTerm', 'auctionFormat', 'series', 'type',
    'securityTermDayMonth', 'securityTermWeekYear', 'term',
    'originalSecurityTerm', 'firstInterestPeriod', '_security_type' ]
```

```
str_cols = [
    'cusip', 'announcedCusip', 'corpusCusip', 'originalCusip',
    'tintCusip1', 'tintCusip2', 'cpiBaseReferencePeriod',
    'pdfFilenameAnnouncement', 'pdfFilenameCompetitiveResults',
    'pdfFilenameNoncompetitiveResults', 'pdfFilenameSpecialAnnouncement',
    'xmlFilenameAnnouncement', 'xmlFilenameCompetitiveResults',
    'xmlFilenameSpecialAnnouncement', 'closingTimeCompetitive',
    'closingTimeNoncompetitive' ]
```

```
a = len(bool_cols) + len(date_cols) + len(int_cols) + len(float_cols) + len(

print(a)
```

```
121
```

```
df_clean = df_raw.dropna(axis=1, how="all")
df_clean = df_clean.loc[:, (df_clean != "").any()]
df_clean = df_clean[df_clean["type"].isin(["Bill", "Note"])].reset_index(dro

df_typed = df_clean.copy()

for col in date_cols:
    if col in df_typed.columns:
        df_typed[col] = pd.to_datetime(df_typed[col], errors='coerce')

for col in bool_cols:
    if col in df_typed.columns:
        df_typed[col] = df_typed[col].map({'Yes': True, 'No': False})

for col in int_cols:
    if col in df_typed.columns:
        df_typed[col] = pd.to_numeric(df_typed[col].replace('', float('nan')

for col in float_cols:
    if col in df_typed.columns:
        df_typed[col] = pd.to_numeric(df_typed[col].replace('', float('nan')

for col in cat_cols:
    if col in df_typed.columns:
        df_typed[col] = df_typed[col].astype('category')
```

Pattern Recognition

```
df = df_typed

df['auctionMonth'] = df['auctionDate'].dt.to_period('M')

bills_by_month = ( df[df['type'] == 'Bill'].groupby('auctionMonth', observed
notes_by_month = ( df[df['type'] == 'Note'].groupby('auctionMonth', observed

rate_by_month = pd.DataFrame({
```

```
    'Bill (highDiscountRate)': bills_by_month,
    'Note (highYield)':        notes_by_month })

print("Average High Rate/Yield by Month (%)")
print(rate_by_month.to_string())
```

```
Average High Rate/Yield by Month (%)
              Bill (highDiscountRate)  Note (highYield)
auctionMonth
2022-01                         0.193             1.450
2022-02                         0.375             1.767
2022-03                         0.486             2.220
2022-04                         0.823             2.747
2022-05                         1.068             2.757
2022-06                         1.595             3.118
2022-07                         2.309             2.932
2022-08                         2.647             3.125
2022-09                         3.047             3.862
2022-10                         3.780             4.185
2022-11                         4.192             4.223
2022-12                         4.217             3.997
```

Pattern 1: Bills and Notes saw their rates rise sharply and consistently throughout 2022.

Bills went from 0.193% to 4.217%, whereas Notes went from $1.450\% to$3.997%$

```
btc_by_term = (df.groupby(['type', 'securityTerm'], observed=True)['bidToCov
    .agg(['mean', 'min', 'max', 'std']).round(3).rename(columns={
        'mean': 'avg_BTC',
        'min':  'min_BTC',
        'max':  'max_BTC',
        'std':  'std_BTC' }) )

print("Bid-to-Cover Ratio by Security Type and Term")
print(btc_by_term.to_string())
```

```
Bid-to-Cover Ratio by Security Type and Term
                      avg_BTC  min_BTC  max_BTC  std_BTC
type securityTerm
Bill 13-Week            2.750     2.27     3.27    0.225
     17-Week            3.017     2.66     3.24    0.171
     26-Week            2.907     2.44     3.32    0.223
     4-Week             2.888     2.35     3.53    0.319
     52-Week            3.015     2.68     3.31    0.216
     8-Week             2.928     2.35     3.70    0.353
Note 10-Year            2.482     2.23     2.68    0.187
     2-Year             2.608     2.46     2.81    0.107
     3-Year             2.495     2.39     2.59    0.063
     5-Year             2.418     2.27     2.53    0.089
     7-Year             2.481     2.33     2.69    0.119
     9-Year 10-Month    2.405     2.34     2.51    0.082
     9-Year 11-Month    2.390     2.31     2.47    0.067
```

Pattern 2: Bid-to-Cover ratio ($= \frac{Received}{Accepted}$) remained above 2.0 for both security types across all of 2022.

This shows that despite the rising rate environment, the investor demand remained strong and consistent.

```
monthly_issuance = ( df.groupby(['auctionMonth', 'type'], observed=True)['of

monthly_issuance['Total'] = monthly_issuance.sum(axis=1)

print("Monthly Total Offering Amount ($ billions)")
print(monthly_issuance.to_string())
```

```
Monthly Total Offering Amount ($ billions)
type            Bill   Note   Total
auctionMonth
2022-01        949.0  250.0  1199.0
2022-02        828.0  242.0  1070.0
2022-03        836.0  230.0  1066.0
2022-04        705.0  221.0   926.0
2022-05        729.0  218.0   947.0
2022-06        707.0  210.0   917.0
2022-07        766.0  205.0   971.0
2022-08        934.0  203.0  1137.0
2022-09        893.0  196.0  1089.0
2022-10       1049.0  192.0  1241.0
2022-11       1100.0  195.0  1295.0
2022-12       1012.0  192.0  1204.0
```

Pattern 3: Bills consistently dominated total issuance

The amount of Bills issued consistenly accounted for more than 75% of the monthly total issuance. We also note a surge in Bills towards the end of the year (from October 2022 onwards). On the contrary, Notes showed a clear declining trend. As rates were going up, it would make sense to scale back on longer-term borrowing.

```
avg_per_auction = ( df.groupby(['type', 'securityTerm'], observed=True)['off
    .rename(columns={'offeringAmount': 'avg_offering_bn'}).sort_values(['typ

print("Average Offering Amount per Auction by Term ($ billions)")
print(avg_per_auction.to_string(index=False))
```

```
Average Offering Amount per Auction by Term ($ billions)
type    securityTerm  avg_offering_bn
Bill         13-Week            54.06
Bill          4-Week            46.73
Bill         26-Week            45.23
Bill          8-Week            40.58
Bill         52-Week            34.00
Bill         17-Week            33.00
Note          5-Year            47.25
Note          2-Year            46.25
Note          3-Year            44.25
```

```
Note          7-Year            41.00
Note         10-Year            35.75
Note 9-Year 10-Month            33.75
Note 9-Year 11-Month            32.75
```

Pattern 4: Despite higher frequency, Bill auctions are comparable in size to Note auctions.

Moreover, we also note that longer-term Notes had the smallest average auction sizes (between 32 and 36), reflecting lower demand due to higher duration risk when rates increase.